

WarrantyHub Frontend Documentation

1. Introduction

WarrantyHub is a comprehensive web application designed to help users manage their product warranties, devices, and maintenance records efficiently. It provides a user-friendly interface for tracking warranty expiration dates, storing important documents, and logging maintenance activities. The application is built with modern web technologies to ensure a responsive and intuitive user experience.

2. Project Structure

The project follows a standard React application structure, organized into logical directories to enhance maintainability and scalability. Below is an overview of the main directories and their contents:

- `public/` : Contains static assets like `index.html` and other public files.
- `src/` :
 - `assets/` : Stores static assets such as images, icons, and fonts.
 - `components/` : Reusable UI components used throughout the application. This directory is further subdivided into:
 - `layout/` : Components related to the overall page layout (e.g., `Layout.jsx` , `Sidebar.jsx` , `Navbar.jsx`).
 - `ui/` : Generic, presentational UI components (e.g., `Button.jsx` , `Input.jsx` , `Modal.jsx`).
 - Other specific components like `ErrorBoundary.jsx` .
 - `contexts/` : React Context API providers for global state management (e.g., `AuthContext.jsx` , `NotificationContext.jsx`).

- `hooks/` : Custom React hooks for encapsulating reusable logic (e.g., `useAuth.js` , `useDevices.js`).
- `pages/` : Contains the main application pages, often corresponding to routes. This is further subdivided into:
 - `auth/` : Authentication-related pages (e.g., `Login.jsx` , `Register.jsx` , `ForgotPassword.jsx`).
 - Other core pages like `Dashboard.jsx` , `Devices.jsx` , `AddDevice.jsx` , `DeviceDetail.jsx` , `Profile.jsx` , `NotFound.jsx` .
- `services/` : Modules responsible for interacting with the backend API (e.g., `api.js` , `authService.js` , `deviceService.js` , `fileService.js`).
- `types/` : TypeScript type definitions for data structures (e.g., `device.ts`).
- `utils/` : Utility functions and helper modules (e.g., `constants.js` , `dateUtils.js` , `formatters.js` , `warrantyUtils.js`).
- `App.jsx` : The main application component, responsible for defining routes and global providers.
- `main.jsx` : The entry point of the React application, rendering the `App` component.
- `index.css` : Global CSS styles, primarily for Tailwind CSS configuration.
- `tailwind.config.js` : Tailwind CSS configuration file.
- `vite.config.js` : Vite build tool configuration file.
- `package.json` : Project metadata and dependencies.

3. Core Technologies

WarrantyHub leverages a modern and robust set of frontend technologies to deliver a high-quality user experience:

React

- **Purpose:** A JavaScript library for building user interfaces. It allows developers to create complex UIs from small and isolated pieces of code called

components.

- **Usage in WarrantyHub:** Used for building all UI components, managing component-based state, and rendering dynamic content.

React Router

- **Purpose:** A standard library for routing in React. It enables navigation between different views in a React application, allowing for a single-page application experience with multiple logical 'pages'.
- **Usage in WarrantyHub:** Manages client-side routing, defining public and protected routes, and handling navigation between pages like Dashboard, Devices, Login, and Register.

React Query (TanStack Query)

- **Purpose:** A powerful data-fetching library for React. It provides hooks for fetching, caching, synchronizing, and updating asynchronous data, making server state management much easier.
- **Usage in WarrantyHub:** Used extensively for managing API calls to the backend, caching responses, and ensuring data consistency across the application. This reduces the need for complex global state management for server-side data.

Tailwind CSS

- **Purpose:** A utility-first CSS framework for rapidly building custom designs. Instead of predefined components, it provides low-level utility classes that can be composed to build any design directly in your markup.
- **Usage in WarrantyHub:** Used for all styling, enabling rapid UI development and ensuring a consistent design system throughout the application. It allows for responsive design and easy customization.

React Hot Toast

- **Purpose:** A lightweight and customizable toast notification library for React applications.
- **Usage in WarrantyHub:** Provides user feedback for various actions, such as successful logins, error messages, and other important notifications.

Vite

- **Purpose:** A next-generation frontend tooling that provides an extremely fast development experience. It serves your source code over native ESM, with a lightning-fast Hot Module Replacement (HMR).
- **Usage in WarrantyHub:** Used as the build tool and development server, offering quick startup times and instant code updates during development.

4. Application Flow and Features

4.1. Authentication

WarrantyHub provides a complete authentication system, allowing users to securely register, log in, and manage their accounts.

- **Login (/login):** Users can sign in with their email and password. The `AuthContext` handles the authentication state, and `authService.js` interacts with the `POST /auth/login` API endpoint. Upon successful login, the user is redirected to the dashboard.
- **Registration (/register):** New users can create an account by providing their name, email, and password. `authService.js` sends the registration data to the `POST /auth/register` endpoint.
- **Forgot Password (/forgot-password):** Users can request a password reset link via email. This interacts with the `POST /auth/forgot-password` endpoint.
- **Reset Password (/reset-password):** After receiving a reset link, users can set a new password using a token. This interacts with the `POST /auth/resetpassword` endpoint.

- **Profile Management (/profile):** Authenticated users can view and update their profile information. `authService.js` handles fetching (`GET /auth/profile`) and updating (`PUT /users/profile`) user data.

4.2. Dashboard

- **Page:** `/dashboard` (`src/pages/Dashboard.jsx`)
- **Purpose:** Provides an overview of the user's warranty and device information. It typically displays key statistics and upcoming warranty expirations.
- **Data Flow:** Fetches device statistics using `deviceService.js` (e.g., `GET /devices/stats`).

4.3. Device Management

This section covers the core functionality of managing devices within the application.

- **View Devices (/devices):** Displays a list of all devices registered by the user. Users can search, filter, and view summary information for each device.
 - **Data Flow:** `deviceService.js` fetches all devices using `GET /devices` and can also handle search queries via `GET /devices/search` .
- **Add Device (/devices/new):** Allows users to add new devices to their inventory. A form collects details such as device name, category, purchase date, and warranty end date.
 - **Data Flow:** `deviceService.js` sends the new device data to `POST /devices` .
- **Device Detail (/devices/:id):** Provides a detailed view of a specific device, including its warranty information, associated documents, and maintenance history. Users can also update or delete the device from this page.
 - **Data Flow:** Fetches device details using `GET /devices/:id` . Updates are sent via `PUT /devices/:id` and deletions via `DELETE /devices/:id` .

4.4. Document Management

Users can upload, view, and manage documents associated with each device, such as purchase receipts or warranty cards.

- **Upload Document:** Users can upload files related to a specific device. The `fileService.js` handles the file upload process.
 - **Data Flow:** `deviceService.js` uses `POST /devices/:id/documents` to upload documents.
- **View Documents:** Lists all documents linked to a device.
 - **Data Flow:** `deviceService.js` fetches documents using `GET /devices/:id/documents` .
- **Download Document:** Allows users to download previously uploaded documents.
 - **Data Flow:** `deviceService.js` handles the download via `GET /devices/:id/documents/:documentId` .
- **Delete Document:** Enables users to remove documents from a device.
 - **Data Flow:** `deviceService.js` sends a request to `DELETE /devices/:id/documents/:documentId` .

4.5. Maintenance Management

Users can log and track maintenance records for their devices.

- **View Records:** Displays a list of all maintenance records for a specific device.
 - **Data Flow:** `deviceService.js` fetches records using `GET /devices/:id/maintenance` .
- **Add Record:** Allows users to add new maintenance entries, including date, description, and cost.
 - **Data Flow:** `deviceService.js` sends new record data to `POST /devices/:id/maintenance` .
- **Update Record:** Enables editing of existing maintenance records.
 - **Data Flow:** `deviceService.js` updates records via `PUT /devices/:id/maintenance/:recordId` .
- **Delete Record:** Allows removal of maintenance records.
 - **Data Flow:** `deviceService.js` sends a request to `DELETE /devices/:id/maintenance/:recordId` .

5. Global State Management and Contexts

WarrantyHub utilizes React's Context API for managing global state, ensuring that critical information is accessible across different components without prop drilling.

AuthContext (`src/contexts/AuthContext.jsx`)

- **Purpose:** Manages the authentication state of the user, including login status, user information, and authentication-related actions.
- **Providers:** Wraps the entire application (or relevant parts) to provide authentication context to all child components.
- **Key State:** `isAuthenticated` (boolean), `user` (object), `isLoading` (boolean).
- **Actions:** `login` , `logout` , `register` , `checkAuthStatus` .
- **Usage:** Components can consume this context using the `useAuth` custom hook (`src/hooks/useAuth.js`) to access authentication status and user data.

NotificationContext (`src/contexts/NotificationContext.jsx`)

- **Purpose:** Provides a centralized way to manage and display application-wide notifications (toasts).
- **Providers:** Wraps the application to make notification functionality available globally.
- **Usage:** Components can trigger toast messages using functions provided by this context, typically via `react-hot-toast` .

6. Custom Hooks

Custom hooks are used to encapsulate reusable logic, making components cleaner and more focused on rendering UI.

`useAuth` (`src/hooks/useAuth.js`)

- **Purpose:** A custom hook that provides convenient access to the `AuthContext` . It abstracts away the direct consumption of the context.
- **Usage:** Used in components that need to interact with authentication state or actions (e.g., `Login.jsx` , `ProtectedRoute.jsx`).

`useDevices` (`src/hooks/useDevices.js`)

- **Purpose:** Likely a custom hook for fetching and managing device-related data using React Query. It would encapsulate the logic for interacting with `deviceService.js` .
- **Usage:** Used in pages like `Devices.jsx` and `DeviceDetail.jsx` to fetch, create, update, and delete device data.

`useFileUpload` (`src/hooks/useFileUpload.js`)

- **Purpose:** A custom hook to handle file upload logic, potentially including progress tracking and error handling.
- **Usage:** Used in components where file uploads are required, such as `DeviceDetail.jsx` for uploading warranty documents.

7. API Services

The `services` directory contains modules responsible for all interactions with the backend API. They abstract away the details of HTTP requests and provide a clean interface for components to fetch and send data.

`api.js`

- **Purpose:** Configures the Axios HTTP client. It sets the base URL for API requests and includes request/response interceptors for handling authentication tokens and global error management.
- **Interceptors:**

- **Request Interceptor:** Automatically attaches the authentication token (from `localStorage`) to outgoing requests.
- **Response Interceptor:** Handles common API error responses (e.g., 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Server Error) by displaying toast messages and redirecting to the login page if the session expires.

authService.js

- **Purpose:** Provides functions for all authentication-related API calls.
- **Methods:** `login` , `register` , `logout` , `getProfile` , `updateProfile` , `forgotPassword` , `resetPassword` , `getToken` , `isAuthenticated` .
- **Endpoints:** Interacts with `/auth/*` and `/users/profile` endpoints.

deviceService.js

- **Purpose:** Provides functions for all device-related API calls, including device CRUD operations, category fetching, search, statistics, and status updates.
- **Methods:** `getAllDevices` , `getDeviceById` , `createDevice` , `updateDevice` , `deleteDevice` , `getCategories` , `searchDevices` , `getDeviceStats` , `updateDeviceStatus` .
- **Endpoints:** Interacts with `/devices/*` endpoints.

fileService.js

- **Purpose:** Handles file-specific operations, likely used for direct file uploads or downloads if not fully encapsulated within `deviceService.js` .
- **Methods:** (Based on `deviceService.js` analysis, file upload/download is handled within `deviceService.js` for device-specific documents. `fileService.js` might be a placeholder or used for more generic file operations not tied to devices.)

8. Utility Functions

The `utils` directory contains various helper functions that provide common functionalities used across the application.

`constants.js`

- **Purpose:** Stores application-wide constants, such as API base URLs, magic numbers, or configuration values.

`dateUtils.js`

- **Purpose:** Provides utility functions for date formatting, parsing, and manipulation.

`formatters.js`

- **Purpose:** Contains functions for formatting data for display, such as currency, numbers, or text.

`warrantyUtils.js`

- **Purpose:** Specific utility functions related to warranty calculations or status checks.

9. Conclusion

This documentation provides a comprehensive overview of the WarrantyHub frontend application, covering its structure, core technologies, application flow, state management, custom hooks, API services, and utility functions. This information should serve as a valuable resource for understanding, maintaining, and extending the application.